



Curso: IF0004 - Desarrollo de Software II

Docente: Angélica Ulate Céspedes

Práctica teórica

Esta práctica tiene como objetivo repasar los conceptos de clases abstractas, herencia, interfaces, enumeraciones, polimorfismo y conceptos vistos en clase. ~~No compila por que 2 metodos tienen el mismo nombre en la clase Vehiculo y Coche~~

Ejercicio #1

Which of the following can fill in the blank in this code to make it compile? (Choose all that apply)

```
public class Ant {  
    _____ void method() { }  
}
```

1. default
2. final
3. private
4. Public
5. String
6. zzz:

Ejercicio #2

. Which of the following compile? (Choose all that apply)

1. final static void method4() { }
2. public final int void method() { }
3. private void int method() { }
4. static final void method3() { }
5. void final method() { }
6. void public method() { }



Ejercicio #3

What modifiers are implicitly applied to all interface methods? (Choose all that apply)

1. protected
2. public
3. static
4. void
5. abstract
6. default

No compila por que 2 metodos tienen el mismo nombre en la clase Vehiculo y Coche

Ejercicio #4

What is a constructor in Java?

- ☐ A method that performs specific tasks.
- ☐ A blueprint for creating classes.
- ☒ A special type of method used to initialize objects.

Ejercicio #5

How is a constructor defined in Java?

- ☒ It has the same name as the class and no return type.
- ☐ It has a different name from the class and a return type.
- ☐ It has a void return type and the same name as the class.

Ejercicio #6



What is the purpose of the `this` keyword in a constructor?

- ☒ To refer to the class itself.
- ☐ To refer to the current object of the class.
- ☐ To call methods from other classes.

No compila por que 2 metodos tienen el mismo nombre en la clase Vehiculo y Coche

Ejercicio # 7

What will be the output of the following code?

```
1 | class Person {
2 |     private String name;
3 |     private int age;
4 |
5 |     public Person(String name, int age) {
6 |         this.name = name;
7 |         this.age = age;
8 |     }
9 | }
10 |
11 | public class Main {
12 |     public static void main(String[] args) {
13 |         Person person = new Person("John", 30);
14 |         System.out.println(person.name + " is " + person.age + " years
15 |         old.");
16 |     }
17 | }
```

- ☐ John is 30 years old.
- ☐ null is 0 years old.
- ☒ An error

Ejercicio # 8



What is a parameterized constructor?

- ☐ A constructor that takes no arguments.
- ☐ A constructor that is automatically provided by Java if no other constructors are defined.
- ☒ A constructor that takes arguments to initialize instance variables with given values.

Ejercicio # 9

What is a default constructor?

- ☐ A constructor that takes arguments to initialize instance variables with given values.
- ☒ A constructor that is automatically provided by Java if no other constructors are defined.
- ☐ A constructor that can only be called from within the class.

Ejercicio # 10



What will be the output of the following code?

```
1 | public class Car {
2 |     private String make;
3 |     private String model;
4 |     private int year;
5 |
6 |     public Car(String make, String model, int year) {
7 |         this.make = make;
8 |         this.model = model;
9 |         this.year = year;
10 |    }
11 | }
12 |
13 | public class Main {
14 |     public static void main(String[] args) {
15 |         Car myCar = new Car("Toyota", "Corolla", 2020);
16 |         System.out.println(myCar.make + " " + myCar.model + " " +
17 |                               myCar.year);
18 |     }
19 | }
```

☐ Toyota Corolla 2020

☐ null null 0

☒ An error

Ejercicio # 11

What is constructor overloading?

☐ Having more than one constructor in a class with the same parameter list.

☒ Having more than one constructor in a class with different parameter lists.

☐ Having a constructor that calls methods from other classes.

Ejercicio # 12



What will be the output of the following code?

```
1 | class Rectangle {  
2 |     int width;  
3 |     int height;  
4 |  
5 |     public Rectangle() {  
6 |         this(1, 1);  
7 |     }  
8 |  
9 |     public Rectangle(int width, int height) {  
10 |         this.width = width;  
11 |         this.height = height;  
12 |     }  
13 | }  
14 |  
15 | public class Main {  
16 |     public static void main(String[] args) {  
17 |         Rectangle rect = new Rectangle();  
18 |         System.out.println(rect.width + " " + rect.height);  
19 |     }  
20 | }
```

☒ 11

☐ 00

☐ An error

Ejercicio #13

What is constructor chaining?

☐ Calling methods from within a constructor.

☒ Calling one constructor from another within the same class using the `this` keyword.

☐ Calling a constructor from a different class.

Ejercicio #14



What is a class in Java?

- ☐ A single instance of an object.
- ☒ A blueprint for creating objects.
- ☐ A method that performs a specific task.

Ejercicio #15

How do you define a class in Java?

☐

```
1 | define class ClassName {  
2 |     // class body  
3 | }
```

☒

```
1 | public class ClassName {  
2 |     // class body  
3 | }
```

☐

```
class ClassName();
```

Ejercicio #16

What is an object in Java?

- ☐ A blueprint for creating classes.
- ☒ An instance of a class.
- ☐ A method that performs a specific task.

Ejercicio #17



What is a method in Java?

☒ A block of code that performs a specific task.

☐ A blueprint for creating objects.

☐ A variable that stores data for an object.

Ejercicio #18

What are fields in a Java class?

☐ Blocks of code that perform a specific task.

☒ Variables that store data for each object of a class.

☐ Methods that define the behavior of an object.

Ejercicio #19

How do you call a method named `honk` on an object `myCar`?

☒ `myCar.honk();`

☐ `honk.myCar();`

☐ `honk();`

Ejercicio #20



What will be the output of the following code?

```
1 | public class Car {  
2 |     void honk() {  
3 |         System.out.println("Beep! Beep!");  
4 |     }  
5 | }  
6 |  
7 | public class Main {  
8 |     public static void main(String[] args) {  
9 |         Car myCar = new Car();  
10 |        myCar.honk();  
11 |    }  
12 | }
```

☐ No output

☒ Beep! Beep!

☐ An error

Ejercicio #21

How do you create an object in Java?

☒ Using the `new` keyword and the constructor of the class.

☐ Using the `create` keyword and the class name.

☐ Using the `make` keyword and the constructor of the class.

Ejercicio #22

What is the syntax for creating an object in Java?

☒ `ClassName objectName = new ClassName();`

☐ `ClassName objectName = ClassName();`

☐ `objectName = new ClassName();`



Ejercicio #23

Which of the following is an example of creating an object in Java?

☒ `Person person1 = new Person();`

☐ `Person person1 = Person();`

☐ `person1 = new Person();`

Ejercicio #24

What is the relationship between a class and an object?

☐ A class is an instance of an object.

☒ An object is an instance of a class.

☐ A class and an object are the same.

Ejercicio #25

What will be the output of the following code?

```
1 | class Dog {  
2 |     void bark() {  
3 |         System.out.println("Woof!");  
4 |     }  
5 | }  
6 |  
7 | public class Main {  
8 |     public static void main(String[] args) {  
9 |         Dog myDog = new Dog();  
10 |        myDog.bark();  
11 |    }  
12 | }
```

☐ No output

☐ An error

☒ Woof!



Ejercicio # 26

What is inheritance in Java?

- ☒ A way to create a new class based on an existing class, inheriting its attributes and methods.
- ☐ A way to convert a class into an interface.
- ☐ A way to create multiple classes with the same name.

Ejercicio #27

How is inheritance implemented in Java?

- ☐ Using the `implements` keyword.
- ☒ Using the `extends` keyword.
- ☐ Using the `inherits` keyword.

Ejercicio # 28



What is the result of the following code?

```
1 | class Vehicle {  
2 |     void start() {  
3 |         System.out.println("Vehicle started");  
4 |     }  
5 | }  
6 |  
7 | class Car extends Vehicle {  
8 |     void openDoor() {  
9 |         System.out.println("Car door opened");  
10 |    }  
11 | }  
12 |  
13 | public class Main {  
14 |     public static void main(String[] args) {  
15 |         Car myCar = new Car();  
16 |         myCar.start();  
17 |         myCar.openDoor();  
18 |     }  
19 | }
```

☒ 1 | Vehicle started
2 | Car door opened

☐ 1 | Car started
2 | Vehicle door opened

☐ An error

Ejercicio #29

What is method overriding in inheritance?

☐ Creating a method in the child class with a different name from any method in the parent class.

☐ Creating multiple methods with the same name in the same class.

☒ Creating a new method in the child class with the same name and parameters as a method in the parent class to provide a specific implementation.

Ejercicio #30



What will be the output of the following code?

```
1 | class Animal {  
2 |     void makeSound() {  
3 |         System.out.println("Animal makes sound");  
4 |     }  
5 | }  
6 |  
7 | class Dog extends Animal {  
8 |     @Override  
9 |     void makeSound() {  
10 |         System.out.println("Dog barks");  
11 |     }  
12 | }  
13 |  
14 | public class Main {  
15 |     public static void main(String[] args) {  
16 |         Dog myDog = new Dog();  
17 |         myDog.makeSound();  
18 |     }  
19 | }
```

☐ Animal makes sound

☒ Dog barks

☐ An error

Ejercicio #31

What does the `super` keyword do in inheritance?

☐ It refers to the current instance of the class.

☒ It refers to the parent class and is used to access parent class methods and constructors.

☐ It creates a new instance of the parent class.

Ejercicio #32



What is the purpose of the `@Override` annotation?

- ☐ To indicate that a method is overloading another method in the parent class.
- ☐ To indicate that a method is abstract.
- ☒ To indicate that a method is overriding a method in the parent class.

Ejercicio #33

What is the "is-a" relationship in inheritance?

- ☐ A relationship where a class is composed of multiple objects.
- ☐ A relationship where a class implements an interface.
- ☒ A relationship where a class inherits from another class, indicating that the subclass is a type of the superclass.

Ejercicio #34



What will be the output of the following code?

```
1 | class Parent {  
2 |     void display() {  
3 |         System.out.println("Display method in Parent");  
4 |     }  
5 | }  
6 |  
7 | class Child extends Parent {  
8 |     @Override  
9 |     void display() {  
10 |         System.out.println("Display method in Child");  
11 |     }  
12 |  
13 |     void show() {  
14 |         super.display();  
15 |     }  
16 | }  
17 |  
18 | public class Main {  
19 |     public static void main(String[] args) {  
20 |         Child obj = new Child();  
21 |         obj.show();  
22 |     }  
23 | }
```

☒ Display method in Parent

☐ Display method in Child

☐ An error

Ejercicio # 35

Which access modifiers allow a subclass to access members of its superclass?

☐ private

☐ default

☒ protected and public

Ejercicio #36



What is polymorphism in Java?

- ☐ The ability of a single function, method, or operator to perform different tasks depending on the context in which it is called.
- ☒ The ability to create a new class based on an existing class, inheriting its attributes and methods.
- ☐ The ability to convert one data type into another.

Ejercicio #37

How is polymorphism achieved in Java?

- ☐ Through the use of constructors and destructors.
- ☒ Through inheritance and the use of interfaces and abstract classes.
- ☐ Through the use of static and final methods.

Ejercicio # 38



What will be the output of the following code?

```
1 | class Shape {  
2 |     void draw() {  
3 |         System.out.println("Drawing a shape");  
4 |     }  
5 | }  
6 |  
7 | class Circle extends Shape {  
8 |     @Override  
9 |     void draw() {  
10 |         System.out.println("Drawing a circle");  
11 |     }  
12 | }  
13 |  
14 | public class Main {  
15 |     public static void main(String[] args) {  
16 |         Shape shape = new Circle();  
17 |         shape.draw();  
18 |     }  
19 | }
```

☐ Drawing a shape

☒ Drawing a circle

☐ An error

Ejercicio #39

What does the `instanceof` operator do in Java?

☐ Checks if two objects are equal.

☐ Converts an object to a different type.

☒ Checks if an object is an instance of a particular class or subclass.

Funcione como compilador y determine si corre o no, y qué imprime

Herencia

Pregunta # 1

```
class Animal {  
    String nombre = "Animal";  
  
    void hacerSonido() {  
        System.out.println("Sonido genérico");  
    }  
}
```

```
class Perro extends Animal {  
    String nombre = "Perro";  
  
    void hacerSonido() {  
        super.hacerSonido();  
        System.out.println("Ladrido");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Perro p = new Perro();  
        System.out.println(p.nombre);  
        p.hacerSonido();  
    }  
}
```

No funciona por que existe un metodo que tiene el mismo nombre en la clase Animal y Perro sin la anotacion `@Override`

Pregunta #2

```
class Vehiculo {  
    void mover() {  
        System.out.println("El vehículo se mueve.");  
    }  
}
```

```
class Coche extends Vehiculo {  
    void mover() {  
        System.out.println("El coche avanza sobre ruedas.");  
    }  
}
```

```
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Vehiculo v = new Coche();  
        v.mover();  
    }  
}
```

No compila por que 2 metodos tienen el mismo nombre en la clase Vehiculo y Coche

Interface

Pregunta #1

```
interface Figura {  
  
    void dibujar();  
  
}
```

```
abstract class Circulo implements Figura {  
  
    abstract void calcularArea();  
  
}
```

```
class CirculoConcreto extends Circulo {  
  
    void calcularArea() {  
  
        System.out.println("Área calculada.");  
  
    }  
  
}
```

```
public void dibujar() {
```



```
        System.out.println("Dibujando círculo.");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        CirculoConcreto c = new CirculoConcreto();  
        c.dibujar();  
        c.calcularArea();  
    }  
}
```

No funciona por que la clase circulo no implementa todos los metodos de la interfaz



Enum

Pregunta #1

```
enum Dia {  
    LUNES, MARTES, MIERCOLES, JUEVES, VIERNES;  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Dia hoy = Dia.MIERCOLES;  
  
        switch (hoy) {  
            case LUNES:  
                System.out.println("Inicio de semana.");  
                break;  
            case VIERNES:  
                System.out.println("Fin de semana cercano.");  
                break;  
            case MIERCOLES:  
                System.out.println("Mitad de semana.");  
                break;  
            default:  
                System.out.println("Otro día.");  
        }  
    }  
}
```

No funciona por que en el switch no se acceden a los elementos del enum sino que se escribe directamente la palabra